

Semivra Libellus - Complete Project Overview & Status Report

Generated 2026-07-04 Branch `fixes/prod-hardening` All results in 8 verified by live runs, not quoted from older documents

1. Executive summary

Semivra Libellus is a full-stack business operations system - Point-of-Sale, inventory, and true double-entry accounting in one web app - built for Philippine small businesses and architected for white-label resale. One codebase runs two business modes: **Food & Beverage** (caf POS with QR table ordering) and **Logistics/Distribution** (this deployment, "Kasa Lokal": dispatch, storage-room stock, and a client portal for business customers).

Where it stands: the application is feature-complete for running a single business today. All 339 server tests, all 9 browser end-to-end tests, lint, and the production build pass; a live manual inspection found zero errors and confirmed the UI works on phone, tablet, and desktop. **The code is production-ready - what remains before go-live is configuration and process** (production environment variables, a tested backup, one staging walkthrough), plus finishing multi-tenancy before hosting multiple businesses on one server.

2. The business case (non-technical)

The problem. Small/independent PH operators are stuck between toy POS apps (no real accounting or inventory) and expensive, US-tax-centric, online-only platforms (Square, Toast) that charge per-transaction fees and don't fit Non-VAT/BIR realities. Reconciliation, cost-of-goods, and tax reporting end up manual and error-prone.

The solution. One tablet-first app that unifies the register, recipe-level inventory with expiry tracking, genuine double-entry accounting, shift/cash control, and customer self-ordering - Non-VAT/BIR-aware out of the box, offline-capable, and brandable for resale.

The impact. Every sale, void, expense, and spoilage automatically posts a balanced accounting entry - the books close themselves. Real-time cost and waste visibility, no oversell (stock deduction is race-safe), and export-ready P&L / Balance Sheet without a bookkeeper re-keying anything.

Team & ownership: s3mivra is sponsor, PM, architect, and developer (with AI pairing); key stakeholders are Kasa Lokal staff, an accountant for BIR sign-off, and future white-label clients.

3. Complete feature inventory - plain English

Selling (POS register)

- Search products, apply peso or percent discounts, park and recall orders, give complimentary orders; voids and refunds are owner-gated.
- Checkout: receipt preview, payment-method buttons (cash / e-wallet / bank / delivery apps), quick-cash denomination buttons (PHP 20-PHP 1000), exact/round shortcuts, automatic change calculation, split payments.
- Fulfilment modes: Dine-In, Takeout, Pickup, Manual Delivery, Grab, Foodpanda.
- Orders move through clear stages - Preparing -> Ready -> Delivered/Picked-Up - with partial-delivery support and a dispatch tracker for deliveries. Duplicate-tap protection means a double-pressed "Place Order" can never create two orders.

Customer self-ordering

- **F&B mode:** customers scan a table QR code and order from their phone. Each QR link is single-use and expires, so old links can't be abused. An "order ready" alert fires in the customer's open tab.
- **Logistics mode:** business clients log into their own **client portal** with a username/password and place orders directly; the owner manages client accounts.

Stock & inventory

- Recipes link ingredients to menu items, so every sale automatically deducts the right ingredients - and the math is race-safe (two cashiers can't oversell the last unit).
- Expiry is tracked batch-by-batch, oldest-first (FEFO), with soonest-expiry badges and a watch panel; low-stock thresholds raise alerts; spoilage requires a reason and is journaled.
- Weighted-average cost recalculates on every restock; cost per gram/ml is tracked; purchase-order suggestions show sensible units (kg/L).
- Excel/CSV bulk import and stock-take with a diff preview before anything is applied; end-of-day physical count reconciliation; every manual adjustment posts a balanced accounting entry.

Accounting (the books keep themselves)

- Full double-entry general ledger with a canonical chart of accounts. Every sale, void, spoilage, expense, fund movement, and inventory adjustment posts a **balanced** journal entry automatically - the system refuses to write an unbalanced one.
- Philippine Non-VAT compliance: 3% percentage-tax model, gross-receipts math, BIR-style sequential references.
- Accounts Receivable discipline: only physical cash hits Cash-on-Hand; e-wallet/bank/delivery-app sales sit in A/R until you record the actual settlement/deposit.
- Manual journal entries, expenses (with on-account/A/P support), revolving/petty-cash funds with a required "paid from" source, bank deposits.
- Reports: P&L, Monthly P&L, Balance Sheet, A/R and A/P outstanding, sales by payment method, profit by category, menu engineering, cashier variance - with CSV/Excel/PDF export and live auto-refresh whenever any transaction posts.

Shifts, staff & cash control

- Staff must clock in after login and declare starting cash; end-of-shift is a guided cash count (by bill denomination) with expected-vs-actual variance recorded per cashier.
- Shift history archive, X-Reading (mid-shift summary without closing), bank deposit recording, automatic midnight close (owner can toggle it off), forced cleanup of parked/unfinished orders at day close.
- Staff hours tracking; the owner is excluded from staff statistics so reports reflect actual employees.

Analytics & audit

- Live dashboard: all-time net revenue, best sales day, top-5 best sellers, total inventory value, SKUs tracked, out-of-stock count, daily revenue trend - exportable to PDF.
- Audit report tab and a stock-card trail for every inventory movement.

Owner controls (superadmin)

- User and role management, client portal accounts, QR ordering open/closed toggle, auto-close toggle, void authority, business settings - all gated so staff can't see or change them.

Everyday practicalities

- Works on phones, tablets, and desktops (layouts verified at all three sizes); installable as an app (PWA); keeps the screen awake at the register; **keeps taking orders when the internet drops** and syncs when it returns; every connected device updates live in real time.

4. Technical architecture (for developers)

Layer	Technology
Frontend	React 18 Vite 8 (rolldown) Tailwind CSS v3 lucide-react socket.io-client PWA (service worker, offline queue)

Backend	Node.js Express 4 Socket.io dual-token JWT auth Zod validation pino structured logging optional Sentry
Database	MongoDB Atlas (Mongoose 9) - multi-document transactions on all money/stock routes
Hosting	Backend on Railway, frontend on Vercel; Docker + docker-compose + nginx for self-host
CI/CD	GitHub Actions (lint + test + build, both packages)
Testing	Vitest (339 server tests, 22 files) Playwright E2E (9 browser tests) in-memory MongoDB harness

Key implementation properties

- **Auth:** 15-minute access JWT held in client memory only (never localStorage) + opaque refresh token in an httpOnly/Secure/SameSite cookie, stored sha256-hashed in a TTL `RefreshSession` collection with true server-side revocation (logout, password/role change, account deletion). Origin-allowlist guard compensates for cross-site cookies. Non-rotating refresh (deliberate - rotation caused false reuse-detection under reload spam). bcrypt cost 12.
- **Integrity:** atomic `findOneAndUpdate $inc` stock deduction; atomic order-number sequencing; balanced-journal assertion (throws if debits credits); completed-order immutability; idempotency keys on order placement.
- **Hardening:** Helmet (HSTS preload, nosniff, frameguard), Zod `validate()` middleware stripping unknown keys (mass-assignment defense, 422 field errors), regex-escape on user input (ReDoS/injection closed), rate limits (login 5/15min, orders 60/min, API 300/min), production error-detail suppression, XSS-escaped print templates, graceful shutdown, crash-drain-and-exit handlers.
- **Fail-fast boot:** in production the server refuses to start with a weak `JWT_SECRET` (<32 chars), default `ADMIN_PASS`, or missing `ALLOWED_ORIGINS`.
- **Multi-tenancy (in progress):** Phase 1 foundation, Phase 2a (tenant identity in auth), Phase 2b partial (fallback-safe read scoping by `businessType` + one-time backfill migration on boot) - all non-breaking so far.
- **Client:** route-level lazy loading; heavy libraries (jsPDF, xlsx, html2canvas) code-split and loaded on demand; top-level error boundary; responsive drawer navigation; horizontally scrollable data tables on small screens.
- **Business modes:** `BUSINESS_TYPE` env (`fb / log`) switches login ordering, client-accounts panel, billing numbers, and mode-specific UI; data rows are stamped with `businessType`.

5. Security posture

Hardened across two dedicated passes (June 2026). Closed: localStorage token theft surface (dual-token rewrite), regex injection/ReDoS, mass assignment, error-detail leakage (~87 catch sites gated), print-window XSS, weak bcrypt cost, missing rate limits, x-powered-by disclosure, and both packages' npm audits brought to 0 known production vulnerabilities. Sessions are revocable server-side; secrets are enforced at boot; logs are structured and never include stacks in production responses.

Note for reviewers: the older `AUDIT\_OVERVIEW.md` (2026-06-05) lists "single JWT in localStorage" as an open P1 finding. That was fixed by the dual-token rewrite later the same week - this document reflects the current state.

6. Quality assurance - what protects against future errors

- **339 server tests** across 22 files: unit (ledger math, tax, units, expiry, chart of accounts, authz) and integration (auth flows, critical paths, money movements, tenancy scoping, sockets, fault injection, edge cases, boot fail-fast, migrations) - run against an in-memory MongoDB, so they're safe and deterministic.
- **9 Playwright browser E2E tests:** real Chrome drives the real UI - login, token-not-in-localStorage, silent-

refresh reload survival, reload-spam resilience, cookie-revocation, every dashboard tab opened with zero-console-error assertions, ledger sub-views.

- **One-command safe E2E environment** (added 2026-07-03): `npm run e2e:server` boots the API on a throwaway in-memory database (never the real one), then `npm run e2e` runs the suite.
- **CI:** GitHub Actions runs lint, tests, and builds on every push.
- **Runtime safety nets:** UI error boundary, centralized API error handler, optional Sentry error reporting, `/health` endpoint, structured logs.

7. Documentation & operations kit

Document / tool	Purpose
<code>`MANUAL.md`</code> (+ PDF)	Full user manual: quick-start tutorial + 15-section reference (POS, inventory, EOD, shifts, accounting, settings, offline, troubleshooting, glossary)
<code>`GO_LIVE.md`</code>	Go-live & rollback runbook for the auth-breaking deploy: env vars, staging checklist, rollback matrix
<code>`DEPLOY.md`</code>	Production deployment runbook: day-1 setup, health checks, backups, incident response
<code>`PROJECT_CHARTER.md`</code>	Charter: scope, stakeholders, milestones, risks, definition of done
<code>`AUDIT_OVERVIEW.md`</code>	June security/competitive audit (partly superseded - see 5 note)
<code>`SESSION_REPORT.md`</code>	Detailed engineering changelog of the hardening session
<code>`Makefile`</code> + <code>`scripts/`</code>	<code>`make health/logs/backup/restore`</code> , env setup, Mongo backup/restore scripts
Docker / docker-compose / nginx	Self-host path

8. Actual verified state (2026-07-03/04)

Everything below was executed in this verification session - not quoted from older reports:

Check	Result
Server test suite (auth, money, ledger, tenancy, sockets, fault injection, edge cases)	339/339 passed
Playwright browser E2E (auth flows, every-tab smoke, ledger sub-views, zero-console-error assertions)	9/9 passed
ESLint - client and server	clean
Production build (<code>`vite build`</code>)	clean, well code-split
Live manual run (login -> Orders, Inventory, Ledger, Analytics with real data)	zero console/runtime errors
Responsiveness - 375px phone / 768px tablet / desktop	all layouts verified (drawer nav, scrollable tables, stacked forms)

9. Deployment readiness

Verdict: the code is ready; the environment and process around it still has open items. Nothing in the application blocks go-live for a single business.

Blockers (must do before/at deploy):

1. Production env vars on the API host: strong `JWT_SECRET` (32 chars), `ALLOWED_ORIGINS` (exact frontend origins), `NODE_ENV=production`, strong `ADMIN_PASS`, `MONGO_URI`. The boot guard enforces these - the server will refuse to start otherwise.
2. Take a database backup **and restore it once** to prove backups work.
3. Walk the `GO_LIVE.md` 2 staging checklist once on a test DB: login -> reload persistence -> logout revocation -> staff shift with cash sale -> ledger entry -> void restores stock -> settings gating -> E2E suite green.
4. Merge `fixes/prod-hardening` -> `main`; deploy server, then client; smoke-test production; watch logs 10-15 minutes.

Strongly recommended:

- Set `SENTRY_DSN` so the first real-user error reports itself immediately.
- Clean the local `server/.env` (stray password line; weak dev JWT secret) - cosmetic locally, but avoids copy-paste accidents.
- Announce a short maintenance window: the auth deploy logs all active users out once.
- HTTPS on both frontend and API hosts (the refresh cookie requires it in production).

Rollback safety: all recent schema changes are additive - rolling code back never requires a database rollback.

First-week watch: midnight auto-close ran, backups produced daily, accountant reviews the first journal entries / P&L / Balance Sheet, monitor logs or Sentry.

10. What's still missing

In-flight (partially built)

- **Multi-tenancy - Phase 2b is partial.** Read scoping by tenant exists; full isolation (write scoping everywhere, Socket.io rooms per tenant, per-tenant settings/branding) must be finished before hosting more than one business on a single deployment. Single-business deployments are unaffected today.

Not built yet (roadmap)

- Multi-branch / franchise rollup reporting (the biggest white-label upsell).
- Loyalty / customer CRM / marketing module.
- Hardware certification matrix (ESC/POS thermal printers, cash drawers, barcode scanners) - printing works through the browser, but no formally certified device list.
- Native app-store apps (currently an installable PWA).
- True server-push notifications - the "order ready" alert only fires while the customer's tab is open.
- Automated per-client provisioning for white-label resale.

Engineering debt (works today, raises future cost)

- `AdminDashboard.jsx` is ~4,600 lines; tab extraction is started but the monolith remains. Risk is to iteration speed, not correctness.
- Analytics are computed client-side (fine at current data volume).
- Browser E2E covers auth + all-tab navigation; checkout/void/EOD are covered by server tests but not yet browser-driven.

Business / process

- Live pilot: run Kasa Lokal on it for 2-4 weeks.
- Accountant sign-off on BIR/Non-VAT correctness of the generated books.
- Resale layer: pricing, licensing, support model, per-instance vs multi-tenant hosting decision.

11. Risks & mitigations

Risk	Impact	Mitigation
Auth deploy logs everyone out / cookie misconfig	High	`GO_LIVE.md` walkthrough; correct `ALLOWED_ORIGINS` + HTTPS; additive schema = code-only rollback
BIR/Non-VAT bookkeeping incorrectness	High	Balanced-entry assertions in code; mandatory accountant review of the first month
Single-tenant limits resale	Medium	Per-instance deployments for first clients; finish multi-tenancy as its own project
Delivery/payment channel reconciliation drift	Medium	Non-cash channels book to A/R until explicitly settled
Tablet performance (large admin bundle)	Low	Lazy routes; heavy libs code-split; verified smooth on target hardware class

12. Competitive position

Competitor	Their gap	Semivra's edge
Square / Toast	US-tax-centric, online-dependent, per-transaction fees	PH Non-VAT/BIR native, offline-first, flat cost, no per-transaction cut
Loyverse	Shallow/paywalled inventory & accounting	Built-in double-entry GL + P&L + Balance Sheet, FEFO expiry
StoreHub / SariPOS	SaaS lock-in, limited recipe costing	White-label, self-hostable, recipe-level COGS, full ledger export
Generic POS + spreadsheet	Manual reconciliation	Auto-journaling, EOD reconciliation, full audit trail

Lead messages: "A POS that closes its own books." "BIR-aware, Non-VAT ready out of the box." "Offline never loses a sale." "Ingredient-true costing."

13. Timeline & version history

[Phase 1: Build] [Phase 2: Hardening] [Phase 3: Verify/Pilot] [Phase 4: Launch/Sell]
 DONE DONE IN PROGRESS PENDING

- **v0.1** (April 2026) - Core POS: QR menu, kitchen display, menu builder, order lifecycle, discounts/VAT.
- **v0.2** (May 2026) - ERP upgrade: double-entry accounting, weighted-average inventory, BOM engine, analytics, exports, dual-mode UI.
- **Hardening** (June 2026) - Security passes, dual-token auth rewrite, bug fixes, accounting corrections, operator

features, docs, CI, Vite 8.

- **Current branch** (June-July 2026) - Production hardening buckets A+B, multi-tenancy Phases 1/2a/2b-partial, logistics client portal fixes, full integration test suite (81 -> 339 tests), browser E2E suite, safe E2E harness.
 - **Verification** (2026-07-03/04) - Everything in 8 executed and green.
-

14. Bottom line

Semivra Libellus has crossed from "project" to "product": the features a single business needs - selling, stock, real books, staff and cash control, client self-ordering, reporting - are built, tested by 348 automated checks, and verified error-free in a live browser at every screen size. The path to production is short and procedural: set the environment variables, prove a backup restores, walk the staging checklist once, merge and deploy. After that, the pilot and the accountant's sign-off turn it into a sellable product - and finishing multi-tenancy unlocks the white-label business model it was designed for.

